

Guía ESP32: Blink, Pulsador y LED por WiFi

Sep 23, 2026 · @Felipe Roa

Introducción y objetivos

En esta guía harás tres ejercicios con una ESP32, de menor a mayor dificultad: hacer parpadear un LED, encenderlo con un botón y controlarlo desde una página web usando la red WiFi que crea la propia placa. Es la base para cualquier objeto interactivo que diseñes más adelante: todo dispositivo recibe información del mundo (entrada), decide qué hacer (lógica) y actúa sobre el mundo (salida).

Reglas de trabajo

- **Ejercicios 1 y 2: código escrito a mano.** No copies ni pegues, y no uses IA. Escribir cada línea te obliga a entender qué hace. Los errores de tipeo también enseñan: aprenderás a leer los mensajes del compilador.
- **Ejercicio 3: se permite usar IA.** Pero debes poder explicar cada línea del código que entregues. Si no puedes explicarla, no está terminado.

Al terminar esta guía podrás:

1. Diferenciar una señal digital de una señal análoga.
2. Explicar qué es un pin de entrada (INPUT) y uno de salida (OUTPUT).
3. Usar `pinMode`, `digitalWrite` y `digitalRead` correctamente.
4. Armar en protoboard un circuito con LED, resistencia y pulsador.
5. Crear una red WiFi con la ESP32 y controlar un LED desde el navegador de tu celular.

Conceptos base

Un microcontrolador solo entiende voltajes en sus pines. Cómo lee y escribe esos voltajes depende de dos ideas: el tipo de señal (digital o análoga) y la dirección del pin (entrada o salida).

Señal digital

Una señal digital tiene solo dos estados posibles, como un interruptor de luz. En la ESP32:

Estado	Palabra en el código	Voltaje en el pin	Equivale a
Alto	HIGH	3,3 V	1, encendido, verdadero

Estado	Palabra en el código	Voltaje en el pin	Equivale a
Bajo	LOW	0 V	0, apagado, falso

Ejemplos: un LED encendido o apagado, un botón presionado o suelto, un sensor de movimiento que detecta o no detecta.

Importante: la ESP32 trabaja a **3,3 V**, no a 5 V como el Arduino Uno. Nunca conectes 5 V directamente a un pin de la ESP32: puedes dañarla.

Señal análoga

Una señal análoga puede tomar cualquier valor dentro de un rango, como la perilla del volumen. La ESP32 la lee con `analogRead(pin)` y devuelve un número entre **0 y 4095** (12 bits): 0 equivale a 0 V y 4095 a 3,3 V.

Ejemplos: un potenciómetro, un sensor de luz (LDR), un sensor de temperatura.

Para *simular* una salida análoga (por ejemplo, un LED que se atenúa), la ESP32 usa **PWM**: enciende y apaga el pin miles de veces por segundo, y el promedio se percibe como un nivel intermedio. En esta guía no la usaremos, pero es el paso siguiente natural.

Entrada (INPUT) y salida (OUTPUT)

Cada pin de propósito general (GPIO) puede configurarse en una de dos direcciones:

- **OUTPUT (salida):** la ESP32 *entrega* voltaje al pin para actuar sobre algo. Ejemplo: encender un LED.
- **INPUT (entrada):** la ESP32 *mide* el voltaje del pin para saber qué pasa afuera. Ejemplo: leer si un botón está presionado.

Las tres funciones que usarás

Función	Para qué sirve	Ejemplo
<code>pinMode(pin, modo)</code>	Define si el pin será entrada o salida. Va en <code>setup()</code> .	<code>pinMode(23, OUTPUT);</code>
<code>digitalWrite(pin, valor)</code>	Pone un pin de salida en <code>HIGH</code> o <code>LOW</code> .	<code>digitalWrite(23, HIGH);</code>
<code>digitalRead(pin)</code>	Lee un pin de entrada y devuelve <code>HIGH</code> o <code>LOW</code> .	<code>int estado = digitalRead(4);</code>

Estructura de todo programa

Todo programa de Arduino tiene dos bloques obligatorios:

```
void setup() {
  // Se ejecuta UNA vez, al encender o reiniciar la placa.
  // Aquí se configura: pinMode, WiFi, monitor serie.
}

void loop() {
  // Se repite para siempre, lo más rápido posible.
  // Aquí va el comportamiento: leer, decidir, actuar.
}
```

```
flowchart LR
  A[Encender placa] --> B["setup()<br/>1 vez"]
  B --> C["loop()<br/>se repite"]
  C --> C
```

Piensa en `setup()` como preparar el escenario y en `loop()` como la función que se repite sin fin.

La ESP32 y los materiales

La ESP32 es un microcontrolador con WiFi y Bluetooth integrados. Esta guía asume una placa **ESP32 DevKit V1** (30 o 38 pines), la más común en el mercado.

Pines que usaremos

Pin	Uso en esta guía	Por qué este pin
GPIO 23	Salida hacia el LED	Pin libre, sin funciones especiales al arrancar
GPIO 4	Entrada del pulsador	Pin libre, admite resistencia pull-up interna
GND	Tierra (0 V)	Cierra todos los circuitos
3V3	Alimentación 3,3 V	No se usa en estos ejercicios

El número del código es el número **GPIO**, no la posición física del pin. Busca la serigrafía "D23" o "G23" en tu placa.

Pines que debes evitar:

- **GPIO 6 a 11:** conectados a la memoria interna. Usarlos bloquea la placa.
- **GPIO 34, 35, 36 y 39:** solo sirven como entrada, no pueden encender un LED.

- **GPIO 0, 2, 12 y 15:** afectan el arranque de la placa. Úsalos solo si sabes lo que haces.

Materiales por estudiante o pareja

Material	Cantidad	Nota
Placa ESP32 DevKit V1	1	
Cable micro-USB o USB-C (según placa)	1	Debe transmitir datos , no solo carga
Protoboard	1	La ESP32 ocupa casi todo el ancho de una protoboard chica; usa una grande o dos juntas
LED de 5 mm (cualquier color)	1	
Resistencia de 220 Ω o 330 Ω	1	Colores 220 Ω : rojo, rojo, café
Pulsador de 4 patas (tactile switch)	1	
Cables jumper macho-macho	5	

¿Por qué el LED necesita resistencia?

Un LED casi no ofrece resistencia al paso de corriente. Sin una resistencia en serie, pasa demasiada corriente: el LED se quema y el pin de la ESP32 puede dañarse. La resistencia de 220 Ω limita la corriente a un valor seguro (unos 5 a 6 mA).

¿Cómo sé cuál pata del LED es cuál?

- **Ánodo (+):** pata **larga**. Va hacia el pin GPIO.
- **Cátodo (-):** pata **corta**, del lado plano del borde del LED. Va hacia GND.

Si el LED está al revés, simplemente no enciende. No se daña, pero tampoco funciona.

Preparar el entorno (Arduino IDE)

Haz esto una sola vez por computador, antes del primer ejercicio.

1. Descarga e instala **Arduino IDE 2** desde arduino.cc/en/software.
2. Abre **Archivo** → **Preferencias** (en Mac: **Arduino IDE** → **Settings**). En el campo *URLs adicionales de gestor de placas* pega: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`

3. Abre **Herramientas** → **Placa** → **Gestor de placas**, busca **esp32** e instala el paquete de **Espressif Systems**. Tarda unos minutos.
4. Conecta la ESP32 por USB. Si el computador no la reconoce, instala el driver del chip USB de tu placa: **CP210x** (Silicon Labs) o **CH340**, según lo que diga el chip junto al conector.
5. En **Herramientas** → **Placa** → **esp32** elige **ESP32 Dev Module**.
6. En **Herramientas** → **Puerto** elige el puerto que aparece al conectar la placa (Windows: COM3 , COM4 ...; Mac: /dev/cu.usbserial... o /dev/cu.SLAB...).

Prueba rápida: desconecta la placa y vuelve a mirar la lista de puertos. El que desaparece es el tuyo.

Cómo subir un programa

- ✓ **Verificar:** revisa que el código no tenga errores. Hazlo siempre antes de subir.
- → **Subir:** compila y envía el programa a la placa.
- Si la consola se queda en `Connecting.....__` más de 5 segundos, **mantén presionado el botón BOOT** de la placa hasta que empiece a subir.
- **Monitor serie** (ícono de lupa, arriba a la derecha): muestra los mensajes que envía la placa con `Serial.println()` . Configúralo en **115200 baudios**.

Ejercicio 1: Blink (escrito a mano)

Objetivo: hacer que un LED se encienda 1 segundo y se apague 1 segundo, para siempre.
Conceptos: **OUTPUT** y **digitalWrite**.

Paso 1: conectar el circuito

Con la placa **desconectada** del USB, arma este circuito:

```
flowchart LR
    A["ESP32<br/>GPIO 23"] --> B["Resistencia<br/>220 Ω"]
    B --> C["LED pata larga (+)"]
    C --> D["LED pata corta (-)"]
    D --> E["ESP32<br/>GND"]
```

Desde	Hacia
Pin GPIO 23	Un extremo de la resistencia
Otro extremo de la resistencia	Pata larga del LED (ánodo +)

Desde	Hacia
Pata corta del LED (cátodo -)	Pin GND de la ESP32

La corriente sale del GPIO 23, pasa por la resistencia y el LED, y vuelve por GND. Si el circuito no está cerrado, no pasa corriente y el LED no enciende.

Paso 2: escribir el código

Abre un sketch nuevo (**Archivo → Nuevo**) y escribe esto a mano, incluidos los comentarios:

```
// Ejercicio 1: Blink
// Enciende y apaga un LED cada 1 segundo.

const int PIN_LED = 23;      // Pin donde está conectado el LED
const int TIEMPO = 1000;     // Tiempo en milisegundos (1000 ms = 1 s)

void setup() {
  pinMode(PIN_LED, OUTPUT);  // El pin 23 será una SALIDA
}

void loop() {
  digitalWrite(PIN_LED, HIGH); // 3,3 V en el pin: LED encendido
  delay(TIEMPO);               // Esperar 1 segundo
  digitalWrite(PIN_LED, LOW);  // 0 V en el pin: LED apagado
  delay(TIEMPO);               // Esperar 1 segundo
}
```

Paso 3: entender cada línea

- `const int PIN_LED = 23;` guarda el número del pin en una constante con nombre. Si mueves el LED a otro pin, cambias un solo número. Evita escribir "números mágicos" sueltos en el código.
- `pinMode(PIN_LED, OUTPUT);` avisa a la ESP32 que ese pin *entregará* voltaje.
- `digitalWrite(PIN_LED, HIGH);` pone el pin en 3,3 V. `LOW` lo pone en 0 V.
- `delay(TIEMPO);` detiene el programa durante 1000 ms. Mientras espera, la placa no hace nada más.
- Al terminar `loop()`, vuelve a empezar desde arriba. Por eso el LED parpadea sin fin.

Paso 4: subir y probar

Conecta la placa, presiona **Verificar** y luego **Subir**. El LED debe parpadear una vez por segundo.

Desafíos

- ☐ Haz que el LED parpadee el doble de rápido.
- ☐ Haz que esté encendido 200 ms y apagado 800 ms (usa dos constantes).
- ☐ Programa un **SOS** en código morse: 3 cortos, 3 largos, 3 cortos.

Ejercicio 2: LED con pulsador (escrito a mano)

Objetivo: que el LED se encienda solo mientras mantienes presionado el botón.

Conceptos: **INPUT**, **digitalRead** y la lógica **leer → decidir → actuar**.

Paso 1: conectar el circuito

Mantén el circuito del LED del Ejercicio 1 y agrega el pulsador. Desconecta la placa antes.

Un pulsador de 4 patas tiene las patas unidas de a pares por dentro. Colócalo **atravesando la canaleta central** de la protoboard y usa dos patas **en diagonal**: así te aseguras de que el botón realmente abre y cierra el circuito.

Desde	Hacia
Pin GPIO 4	Una pata del pulsador
Pata diagonal opuesta del pulsador	Pin GND de la ESP32
(LED y resistencia)	Igual que en el Ejercicio 1, en GPIO 23

```

flowchart LR
  A["ESP32<br/>GPIO 4"] --- B[Pulsador]
  B --- C["ESP32<br/>GND"]
  D["ESP32<br/>GPIO 23"] --> E["220 Ω + LED"]
  E --> C
  
```

La resistencia pull-up interna

Si un pin de entrada no está conectado a nada, queda "flotando": lee valores al azar por el ruido eléctrico del ambiente. Para evitarlo, usamos `INPUT_PULLUP`, que activa una resistencia dentro de la ESP32 que mantiene el pin en `HIGH` por defecto.

Esto invierte la lógica, y es lo que más confunde al principio:

Botón	Qué pasa en el pin	digitalRead devuelve
Suelto	La pull-up lo mantiene en 3,3 V	HIGH
Presionado	El botón lo conecta directo a GND	LOW

Presionado = LOW . Recuérdalo al escribir el if .

Paso 2: escribir el código

Sketch nuevo, escrito a mano:

```
// Ejercicio 2: LED con pulsador
// El LED se enciende mientras el botón está presionado.

const int PIN_LED = 23;    // Salida: LED
const int PIN_BOTON = 4;   // Entrada: pulsador

void setup() {
  pinMode(PIN_LED, OUTPUT);    // El LED es una SALIDA
  pinMode(PIN_BOTON, INPUT_PULLUP); // El botón es una ENTRADA con pull-up
  interna
  Serial.begin(115200);        // Para ver mensajes en el Monitor
  serie
}

void loop() {
  // 1. LEER
  int estadoBoton = digitalRead(PIN_BOTON);

  // 2. DECIDIR y 3. ACTUAR
  if (estadoBoton == LOW) {      // LOW = presionado (por la pull-up)
    digitalWrite(PIN_LED, HIGH);
    Serial.println("Botón presionado: LED encendido");
  } else {
    digitalWrite(PIN_LED, LOW);
  }

  delay(10); // Pausa breve para no saturar el Monitor serie
}
```

Paso 3: entender cada línea

- `pinMode(PIN_BOTON, INPUT_PULLUP);` configura el pin 4 como entrada y activa la resistencia interna.
- `int estadoBoton = digitalRead(PIN_BOTON);` lee el pin y guarda el resultado (`HIGH` o `LOW`) en una variable.
- `if (estadoBoton == LOW)` pregunta: ¿está presionado? Ojo: `==` compara, `=` asigna. Confundirlos es un error clásico.
- `Serial.println(...)` envía texto al computador. Abre el Monitor serie a 115200 baudios para verlo. Es tu mejor herramienta para encontrar errores.
- En este ejercicio el `loop()` se repite cientos de veces por segundo. Por eso la respuesta al botón se siente inmediata.

Paso 4: subir y probar

Sube el código. Al presionar el botón, el LED enciende; al soltarlo, se apaga.

Desafíos

- ☐ Invierte el comportamiento: LED encendido siempre, se apaga al presionar.
- ☐ Haz que el LED parpadee mientras el botón está presionado.
- ☐ **Desafío mayor:** que cada pulsación *alterne* el LED (presiono: enciende; presiono otra vez: apaga). Pista: guarda el estado anterior del botón en una variable y actúa solo cuando cambia de `HIGH` a `LOW`. Notarás que a veces falla por el *rebote* mecánico del botón; investiga qué es el *debounce*.

Ejercicio 3: LED controlado desde una página web (IA permitida)

Objetivo: la ESP32 crea su propia red WiFi y muestra una página web con dos botones, Encender y Apagar. Desde tu celular te conectas a esa red y controlas el LED. No necesitas internet ni router.

Paso 1: el circuito

El mismo LED del Ejercicio 1: GPIO 23 → resistencia 220 Ω → LED → GND. El pulsador puede quedarse conectado; este código no lo usa.

Paso 2: conceptos nuevos

- **Punto de acceso (Access Point, AP):** la ESP32 se comporta como un router y crea una red WiFi propia. Su dirección fija es **192.168.4.1**.

- **Servidor web:** un programa que espera pedidos (*requests*) del navegador y responde con una página.
- **Ruta (URL):** cada dirección de la página activa una función distinta. `/` muestra la página, `/encender` y `/apagar` cambian el LED.
- **HTML:** el lenguaje de la página. La ESP32 la guarda como texto y la envía al navegador.

```
sequenceDiagram
    participant C as Celular (navegador)
    participant E as ESP32 (servidor)
    participant L as LED
    C->>E: GET /
    E-->>C: Página HTML con botones
    C->>E: GET /encender (toque en botón)
    E->>L: digitalWrite(23, HIGH)
    E-->>C: Vuelve a la página, estado ENCENDIDO
```

Fíjate que al final todo termina en el mismo `digitalWrite` del Ejercicio 1. La web solo es otra forma de *entrada*, igual que el botón.

Paso 3: usar IA de forma responsable

Puedes pedirle el código a una IA (Claude, ChatGPT, Copilot, etc.). Para que el resultado sirva, tu pedido debe ser específico. Un ejemplo de buen prompt:

Escribe un sketch de Arduino para ESP32 (Arduino IDE, core de Espressif 3.x). La ESP32 debe crear una red WiFi en modo Access Point llamada "ESP32-Grupo01" con clave "diseno2026". Debe levantar un servidor web con la librería WebServer.h en el puerto 80. La ruta "/" muestra una página HTML adaptada a celular con dos botones, Encender y Apagar, y el estado actual del LED. Las rutas "/encender" y "/apagar" controlan un LED en el GPIO 23 con digitalWrite y redirigen a "/". Imprime la IP en el Monitor serie a 115200 baudios. No uses delay() dentro de loop(). Comenta cada bloque en español.

Condiciones para que el ejercicio sea válido:

- ☐ Entregas el prompt (o prompts) que usaste.
- ☐ El LED está en GPIO 23, se usa `pinMode`, `digitalWrite` y constantes con nombre, como en los ejercicios anteriores.
- ☐ El `loop()` no tiene `delay()`: la placa debe estar siempre atenta a nuevos pedidos.
- ☐ Cambiaste el nombre de la red por el de tu grupo, para no confundirte con las demás.
- ☐ Puedes explicar en voz alta qué hace cada línea. El profesor puede preguntarte por cualquiera.

Paso 4: código de referencia

Compara lo que te entregó la IA con esta versión. Si algo difiere, averigua por qué.

```
// Ejercicio 3: LED controlado desde una página web
// La ESP32 crea su propia red WiFi y sirve una página con dos botones.

#include <WiFi.h>           // Funciones de WiFi
#include <WebServer.h>      // Servidor web simple

// ---- Configuración ----
const char* NOMBRE_RED = "ESP32-Grupo01"; // Cambia por el nombre de tu
grupo (sin ñ ni tildes)
const char* CLAVE_RED = "diseno2026";    // Mínimo 8 caracteres
const int PIN_LED = 23;

WebServer servidor(80);    // Servidor en el puerto 80 (HTTP)
bool ledEncendido = false; // Recuerda el estado actual del LED

// Construye la página HTML según el estado del LED
String crearPagina() {
    String estado = ledEncendido ? "ENCENDIDO" : "APAGADO";
    String html = "<!DOCTYPE html><html lang='es'><head>";
    html += "<meta charset='UTF-8'>";
    html += "<meta name='viewport' content='width=device-width, initial-
scale=1'>";
    html += "<title>Control LED ESP32</title>";
    html += "<style>";
    html += "body{font-family:sans-serif;text-
align:center;padding:40px;background:#111;color:#eee;}";
    html += "a{display:block;margin:16px auto;padding:20px;width:200px;border-
radius:12px;";
    html += "font-size:22px;text-decoration:none;color:#fff;}";
    html += ".on{background:#2e7d32;} .off{background:#c62828;}";
    html += "</style></head><body>";
    html += "<h1>LED ESP32</h1>";
    html += "<p>Estado: <strong>" + estado + "</strong></p>";
    html += "<a class='on' href='/encender'>Encender</a>";
    html += "<a class='off' href='/apagar'>Apagar</a>";
    html += "</body></html>";
    return html;
}

// Redirige el navegador de vuelta a la página principal
void volverAlInicio() {
    servidor.sendHeader("Location", "/");
    servidor.send(303);
}
```

```

}

// Ruta "/": muestra la página
void paginaPrincipal() {
    servidor.send(200, "text/html", crearPagina());
}

// Ruta "/encender"
void encenderLed() {
    ledEncendido = true;
    digitalWrite(PIN_LED, HIGH);
    volverAlInicio();
}

// Ruta "/apagar"
void apagarLed() {
    ledEncendido = false;
    digitalWrite(PIN_LED, LOW);
    volverAlInicio();
}

void setup() {
    Serial.begin(115200);
    pinMode(PIN_LED, OUTPUT);
    digitalWrite(PIN_LED, LOW); // Partir con el LED apagado

    // Crear la red WiFi propia (modo Access Point)
    WiFi.softAP(NOMBRE_RED, CLAVE_RED);
    Serial.print("Red creada: ");
    Serial.println(NOMBRE_RED);
    Serial.print("Abre en el navegador: http://");
    Serial.println(WiFi.softAPIP()); // Normalmente 192.168.4.1

    // Asociar cada ruta con su función
    servidor.on("/", paginaPrincipal);
    servidor.on("/encender", encenderLed);
    servidor.on("/apagar", apagarLed);
    servidor.begin();
}

void loop() {
    servidor.handleClient(); // Atender pedidos del navegador. Sin delay().
}

```

Paso 5: subir y probar

1. Sube el código y abre el Monitor serie a 115200 baudios. Verás el nombre de la red y la IP.
2. En tu celular, abre la configuración WiFi y conéctate a la red de tu grupo con la clave.
3. Si el celular avisa "sin conexión a internet", elige **mantener la conexión**. Si no carga, desactiva los datos móviles.
4. Abre el navegador y escribe `http://192.168.4.1`.
5. Toca Encender y Apagar. El LED debe responder en menos de un segundo.

Desafíos

- ☐ Agrega un tercer botón que haga parpadear el LED 5 veces.
- ☐ Rediseña la página: tipografía, colores, un ícono de ampolleta. Eres diseñador o diseñadora de interacción; que se note.
- ☐ Combina con el Ejercicio 2: que el LED se pueda controlar desde el botón físico y desde la web, y que la página muestre el estado correcto en ambos casos.

Problemas frecuentes

Síntoma	Causa probable	Solución
No aparece ningún puerto	Cable solo de carga o falta driver	Prueba otro cable; instala driver CP210x o CH340
Se queda en Connecting....__	La placa no entra en modo carga	Mantén presionado BOOT mientras sube
'PIN_LED' was not declared	Error de tipeo o constante mal escrita	Revisa mayúsculas: <code>PIN_LED</code> y <code>Pin_Led</code> son distintos
expected ';' before...	Falta un punto y coma	Mira la línea anterior a la que indica el error
El LED no enciende nunca	LED al revés o circuito abierto	Pata larga hacia el GPIO; revisa que todo llegue a GND
El LED enciende siempre	Conectado a 3V3 en vez de GPIO 23	Revisa el cable del pin
El botón funciona al revés	Olvidaste que con pull-up presionado = <code>LOW</code>	Revisa la condición del <code>if</code>

Síntoma	Causa probable	Solución
El botón responde al azar	Pin flotando o pulsador mal orientado	Usa <code>INPUT_PULLUP</code> y patas en diagonal
No veo la red WiFi	Clave de menos de 8 caracteres	Usa una clave de 8 o más caracteres
La página no carga	El celular salió de la red por falta de internet	Mantén la conexión y apaga los datos móviles
Monitor serie muestra símbolos raros	Velocidad distinta	Configura 115200 baudios

Entrega

- ☐ **Ejercicio 1:** video corto (10 s) del LED parpadeando + archivo `.ino` escrito a mano.
- ☐ **Ejercicio 2:** video del botón controlando el LED + archivo `.ino` escrito a mano + un desafío resuelto.
- ☐ **Ejercicio 3:** video del celular controlando el LED + archivo `.ino` + los prompts usados + una explicación breve, con tus palabras, de qué hacen `softAP`, `servidor.on` y `handleClient`.

Glosario

Término	Significado
GPIO	<i>General Purpose Input/Output</i> : pin que puede ser entrada o salida
HIGH / LOW	Estado alto (3,3 V) o bajo (0 V) de un pin digital
INPUT / OUTPUT	Pin que mide voltaje / pin que entrega voltaje
INPUT_PULLUP	Entrada con resistencia interna que la mantiene en <code>HIGH</code>
Señal digital	Solo dos valores posibles: 0 o 1
Señal analógica	Cualquier valor dentro de un rango (en ESP32: 0 a 4095)
PWM	Técnica que simula una salida analógica encendiendo y apagando muy rápido

Término	Significado
Sketch	Programa de Arduino (archivo <code>.ino</code>)
Compilar	Traducir el código a instrucciones que entiende la placa
Monitor serie	Ventana para leer mensajes enviados por la placa
Access Point	Modo en que la ESP32 crea su propia red WiFi
Servidor web	Programa que responde páginas cuando el navegador las pide
Ruta	Parte de la dirección web (<code>/encender</code>) que activa una acción